利用条件变量实现线程安全队列

Posted on 2023-09-10 11:28:28 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2VBjkKOuIv1YiZ1nyLKQKQ9wRBA)

简介

本文介绍如何使用条件变量控制并发的同步操作，试想有一个线程A一直输出1，另一个线程B一直输出2。我想让两个线程交替输出1，2，1，2...之类的效果，该如何实现？有的同学可能会说不是有互斥量mutex吗？可以用一个全局变量num表示应该哪个线程输出，比如num为1则线程A输出1，num为2则线程B输出2，mutex控制两个线程访问num，如果num和线程不匹配，就让该线程睡一会，这不就实现了吗？比如线程A加锁后发现当前num为2则表示它不能输出1，就解锁，将锁的使用权交给线程A，线程B就sleep一会。

不良实现

上面说的方式可以实现我们需要的功能，代码如下

```
1. void PoorImpleman() {
2.     std::thread t1([]() {
3.         for (;;) {
4.
5.             {
6.                 std::lock_guard<std::mutex> lock(mtx_num);
7.                 if (num == 1) {
8.                     std::cout << "thread A print 1....." << std::endl;
9.                     num++;
10.                    continue;
11.                }
12.            }
13.
14.            std::this_thread::sleep_for(std::chrono::milliseconds(500));
15.
16.        }
17.
18.    });
19.
20.    std::thread t2([]() {
21.        for (;;) {
22.
23.            {
24.                std::lock_guard<std::mutex> lock(mtx_num);
25.                if (num == 2) {
26.                    std::cout << "thread B print 2....." << std::endl;
27.                    num--;
28.                    continue;
29.                }
30.            }
31.
32.            std::this_thread::sleep_for(std::chrono::milliseconds(500));
33.
34.        }
35.
36.        });
37.
38.    t1.join();
39.    t2.join();
40. }
```

PoorImpleman 虽然能实现我们交替打印的功能，会造成消息处理的不及时，因为线程A要循

环检测 `num` 值，如果 `num` 不为1，则线程A就睡眠了，在线程A睡眠这段时间很可能B已经处理完打印了，此时A还在睡眠，是对资源的浪费，也错过了最佳的处理时机。所以我们提出了用条件变量来通知线程的机制，当线程A发现条件不满足时可以挂起，等待线程B通知，线程B通知A后，A被唤醒继续处理。

条件变量

我们这里用条件变量实现上面的逻辑

```
void ResonableImplemention() {
    std::thread t1([]() {
        for (;;) {

            std::unique_lock<std::mutex> lock(mtx_num);
            cvA.wait(lock, []() {
                return num == 1;
                });

            num++;
            std::cout << "thread A print 1....." << std::endl;
            cvB.notify_one();
        }

        });

    std::thread t2([]() {
        for (;;) {

            std::unique_lock<std::mutex> lock(mtx_num);
            cvB.wait(lock, []() {
                return num == 2;
                });

            num--;
            std::cout << "thread B print 2....." << std::endl;
            cvA.notify_one();
        }

        });

    t1.join();
    t2.join();
}
```

当条件不满足时(num 不等于1 时) `cvA.wait` 就会挂起，等待线程B通知通知线程A唤醒，线程B采用的是 `cvA.notifyone` 。

这么做的好处就是线程交替处理非常及时。比起 `sleep` 的方式，我们可以从控制台上看出差异效果， `sleep` 的方式看出日志基本是每隔1秒才打印一次，效率不高。

线程安全队列

之前我们实现过线程安全的栈，对于pop操作，我们如果在线程中调用empty判断是否为空，如果不为空，则pop，因为empty和pop内部分别加锁，是两个原子操作，导致pop时可能会因为其他线程提前pop导致队列为空，从而引发崩溃。我们当时的处理方式是实现了两个版本的pop，一种是返回智能指针类型，一种通过参数为引用的方式返回。对于智能指针版本我们发现队列为空则返回空指针，对于引用版本，
发现队列为空则抛出异常，这么做并不是很友好，所以我们可以通过条件变量完善之前的程序，不过这次我们重新实现一个线程安全队列。

```
#include <queue>
#include <mutex>
#include <condition_variable>
template<typename T>
class threadsafe_queue
{
private:
    std::mutex mut;
    std::queue<T> data_queue;
    std::condition_variable data_cond;
public:
    void push(T new_value)
    {
        std::lock_guard<std::mutex> lk(mut);
        data_queue.push(new_value);
        data_cond.notify_one();
    }
    void wait_and_pop(T& value)
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk,[this]{return !data_queue.empty();});
        value=data_queue.front();
        data_queue.pop();
    }
};

threadsafe_queue<data_chunk> data_queue;
void data_preparation_thread()
{
    while(more_data_to_prepare())
    {
        data_chunk const data=prepare_data();
        data_queue.push(data);    <---- ②
    }
}
void data_processing_thread()
{
    while(true)
    {
        data_chunk data;
        data_queue.wait_and_pop(data);
        process(data);
```

```
43.         if(is_last_chunk(data))
44.             break;
45.     }
46. }
```

我们可以启动三个线程，一个 `producer` 线程用来向队列中放入数据。一个 `consumer1` 线程用来阻塞等待pop队列中的元素。

另一个 `consumer2` 尝试从队列中pop元素，如果队列为空则直接返回，如果非空则pop元素。

打印时为了保证线程输出在屏幕上不会乱掉，所以加了锁保证互斥输出

测试代码如下

```
1. void test_safe_que() {
2.     threadsafe_queue<int>  safe_que;
3.     std::mutex  mtx_print;
4.      std::thread producer(
5.         [&]() {
6.             for (int i = 0; ;i++) {
7.                 safe_que.push(i);
8.                 {
9.                     std::lock_guard<std::mutex> printlk(mtx_print);
10.                    std::cout << "producer push data is " << i << std::endl;
11.                }
12.
13.                std::this_thread::sleep_for(std::chrono::milliseconds(200));
14.            }
15.        }
16.    );
17.
18.    std::thread consumer1(
19.        [&]() {
20.            for (;;) {
21.                auto data = safe_que.wait_and_pop();
22.                {
23.                    std::lock_guard<std::mutex> printlk(mtx_print);
24.                    std::cout << "consumer1 wait and pop data is " << *data << std::endl;
25.                }
26.
27.                std::this_thread::sleep_for(std::chrono::milliseconds(500));
28.            }
29.        }
30.    );
31.
32.    std::thread consumer2(
33.        [&]() {
34.            for (;;) {
35.                auto data = safe_que.try_pop();
36.                if (data != nullptr) {
37.                    {
38.                        std::lock_guard<std::mutex> printlk(mtx_print);
39.                        std::cout << "consumer2 try_pop data is " << *data << std::endl;
40.                    }
41.
42.                }
```

```
43.
44.                 std::this_thread::sleep_for(std::chrono::milliseconds(500));
45.             }
46.         }
47.     );
48.
49.     producer.join();
50.     consumer1.join();
51.     consumer2.join();
52. }
```

测试效果如下

```
1. producer push data is 0
2. consumer1 wait and pop data is 0
3. producer push data is 1
4. producer push data is 2
5. consumer2 try_pop data is 1
6. consumer1 wait and pop data is 2
7. producer push data is 3
8. producer push data is 4
9. consumer2 try_pop data is 3
10. consumer1 wait and pop data is 4
11. producer push data is 5
12. producer push data is 6
13. producer push data is 7
14. consumer2 try_pop data is 5
15. consumer1 wait and pop data is 6
```

我们能看到consumer1和consumer2是并发消费的

总结

本文介绍了如何通过条件变量实现并发线程的同步处理。

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

源码链接

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2679) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(592) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)